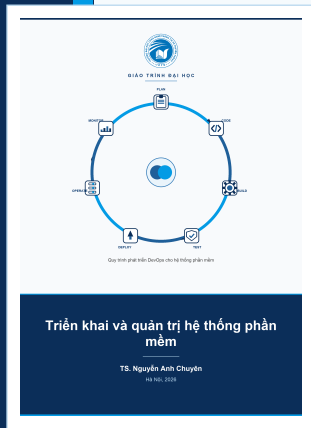




## CHƯƠNG 1

# Tổng quan về triển khai và quản trị hệ thống

Triển khai và quản trị hệ thống phần mềm



# Giới thiệu nội dung

## Mục tiêu chương

- Trình bày khái niệm DevOps, quy trình 7 giai đoạn và tư duy IaC.
- Giải thích vai trò của Linux và cấu hình dịch vụ SSH, FTP cơ bản.
- Giải thích cơ chế Nginx (reverse proxy) và vì sao cần SSL/TLS.
- Triển khai được Nginx reverse proxy kèm HTTPS tự ký bằng OpenSSL.

## Nội dung chính

- 1 DevOps và tư duy hạ tầng IaC**
- 2 Quản trị hệ điều hành Linux**
- 3 Nginx và bảo mật SSL/TLS**
- 4 Thực hành: Reverse Proxy và HTTPS tự ký**

# Vị trí môn học trong lộ trình đào tạo

- Các môn trước tập trung vào **viết mã đúng và hiệu quả**; môn học này chuyển sang **vận hành hệ thống thực tế**.
- Đưa ứng dụng từ môi trường phát triển (Development) ra môi trường sản xuất (Production) trước người dùng thật.
- Trọng tâm chuyển từ tính đúng đắn của logic sang **tính bền bỉ, khả năng mở rộng và an toàn** của dịch vụ.
- Gắn liền với tư duy **DevSecOps**: tích hợp bảo mật vào mọi giai đoạn, không chỉ kiểm tra ở bước cuối.

## Câu hỏi mở đầu

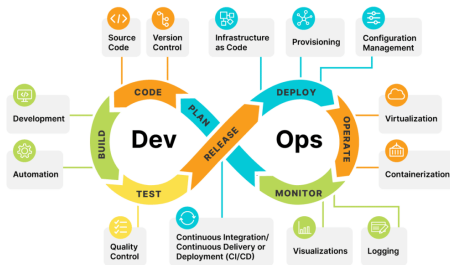
Một hệ thống “chạy được” trên máy cá nhân có chắc chắn “chạy tốt, chạy nhanh, chạy an toàn” khi phục vụ hàng nghìn người dùng thật không?

# DevOps: văn hóa làm việc, không phải công cụ

- **DevOps** = Development (Dev) + Operations (Ops).
- Là văn hóa kết hợp giữa đội ngũ phát triển và đội ngũ vận hành (hệ thống, mạng, bảo mật, hạ tầng).
- Mục tiêu: rút ngắn vòng đời phát triển phần mềm (SDLC – Software Development Life Cycle).
- Không có công cụ hay tiêu chuẩn bắt buộc – DevOps là một **quy trình khép kín** lặp lại liên tục.

# Quy trình DevOps: vòng lặp khép kín 7 giai đoạn

- 1 Plan
- 2 Code
- 3 Build
- 4 Test
- 5 Release & Deploy
- 6 Operate
- 7 Monitor

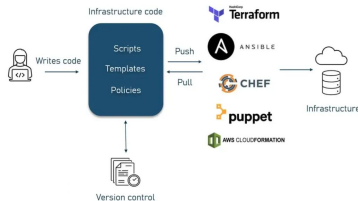


# Công cụ tiêu biểu theo từng giai đoạn

| Giai đoạn           | Công cụ tiêu biểu                 |
|---------------------|-----------------------------------|
| 1. Plan             | Jira, Trello, Slack               |
| 2. Code             | Git, GitHub, GitLab               |
| 3. Build            | Maven, Gradle, Docker             |
| 4. Test             | JUnit, Selenium, Jenkins          |
| 5. Release & Deploy | Jenkins, GitLab CI/CD, Kubernetes |
| 6. Operate          | Terraform, Ansible                |
| 7. Monitor          | Prometheus, Grafana, ELK Stack    |

# Từ thủ công đến khai báo: Infrastructure as Code (IaC)

- **Thủ công (Manual):** đăng nhập từng máy, gõ từng lệnh – dễ sai sót khi nhân rộng ra hàng trăm máy chủ.
- **IaC (Khai báo):** viết tệp mô tả trạng thái mong muốn (thường bằng YAML/JSON), công cụ tự tạo hệ thống đúng như mô tả.



# Ba giá trị cốt lõi của IaC

- **Tính nhất quán (Consistency):** môi trường Staging giống hệt Production.
- **Khả năng tái bản (Repeatability):** sự cố xảy ra – chạy lại mã IaC để dựng lại hệ thống trong phút chốc.
- **Quản lý phiên bản (Version Control):** hạ tầng là mã nguồn, có thể đưa lên Git và rollback khi cần.

## Nguyên lý Hạ tầng bất biến

Máy chủ lỗi – không sửa, mà xóa và tạo lại từ mã nguồn – tránh hiện tượng **Configuration Drift**.

# Vì sao Linux là “ngôn ngữ chung” của Cloud và DevOps?

- Hơn 90% hạ tầng đám mây công cộng (AWS, Azure, Google Cloud) chạy trên Linux.
- Triết lý “**Mọi thứ đều là tệp tin**” – can thiệp sâu vào kernel qua dòng lệnh.
- Docker dựa trên tính năng lõi của Linux Kernel: **Namespaces** (cách ly) và **Cgroups** (giới hạn tài nguyên).
- Có thể chạy chỉ với vài trăm MB RAM (bản Minimal, không GUI) – tiết kiệm chi phí phần cứng.
- Mã nguồn mở, minh bạch – lỗ hổng bảo mật (CVE) thường được vá trong vài giờ.

# Quy trình 4 bước cấu hình dịch vụ trên Linux

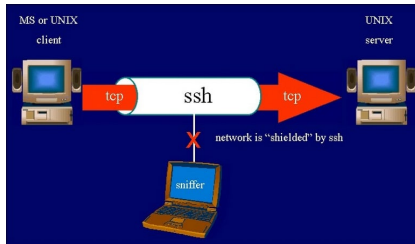
- 1 **Cài đặt (Install):** dùng `apt` hoặc `dnf` để tải gói.
- 2 **Cấu hình (Configure):** chỉnh sửa tệp trong thư mục `/etc`.
- 3 **Kiểm soát trạng thái (Control):** dùng `systemctl` để start/stop/reload.
- 4 **Kiểm tra & Bảo mật (Check & Secure):** xem log tại `/var/log`, mở cổng tường lửa tương ứng.

## Áp dụng

Quy trình này được áp dụng ngay cho hai dịch vụ minh họa tiếp theo: SSH và FTP.

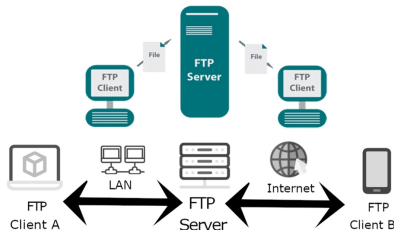
## Ví dụ: dịch vụ SSH (Secure Shell)

- Cho phép quản trị máy chủ từ xa mà không cần ngồi trước màn hình vật lý.
- Mã hóa toàn bộ dữ liệu truyền đi, bảo vệ mật khẩu khỏi nghe lén.
- Xác thực bằng mật khẩu hoặc **cặp khóa công khai/riêng tư** (khuyến nghị).
- Tệp cấu hình:  
`/etc/ssh/sshd_config`.



## Ví dụ: dịch vụ FTP với vsftpd

- VSFTPD (Very Secure FTP Daemon): dịch vụ FTP nhanh, an toàn, phổ biến trên Linux.
- Tập cấu hình chính:  
`/etc/vsftpd.conf`.
- Cần bật `write_enable`, `local_enable`, `chroot_local_user` để cho phép upload an toàn.



### Lưu ý bảo mật

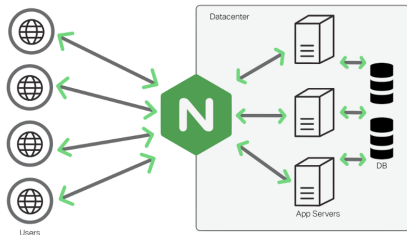
FTP truyền thông gửi mật khẩu dạng văn bản thuần túy. Hệ thống thật nên dùng **SFTP** hoặc **FTPS** để mã hóa.

# Các lệnh quản lý gói cơ bản trên Debian/Ubuntu

| Lệnh                                 | Chức năng                                      |
|--------------------------------------|------------------------------------------------|
| <code>apt update</code>              | Cập nhật danh sách gói, chưa nâng cấp phần mềm |
| <code>apt upgrade</code>             | Nâng cấp phần mềm lên phiên bản mới nhất       |
| <code>apt install &lt;gói&gt;</code> | Cài đặt gói mới cùng các phụ thuộc             |
| <code>apt remove &lt;gói&gt;</code>  | Gỡ phần mềm, giữ lại tệp cấu hình              |
| <code>apt purge &lt;gói&gt;</code>   | Gỡ hoàn toàn, xóa cả tệp cấu hình              |
| <code>apt autoremove</code>          | Dọn các gói phụ thuộc không còn dùng           |
| <code>apt list --installed</code>    | Liệt kê phần mềm đã cài trên hệ thống          |

# Nginx: máy chủ web hướng sự kiện (event-driven)

- Máy chủ web / reverse proxy mã nguồn mở phổ biến nhất hiện nay.
- Kiến trúc **hướng sự kiện**: một tiến trình xử lý hàng ngàn kết nối, khác với Apache (mỗi request một tiến trình).
- Giải quyết bài toán **C10k**: hỗ trợ hơn 10.000 kết nối đồng thời với bộ nhớ thấp.

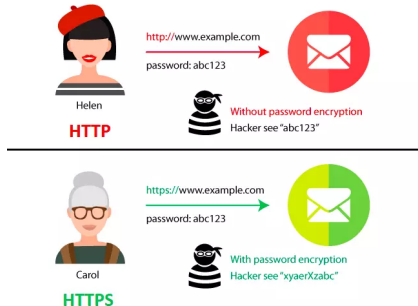


# Bốn vai trò chính của Nginx trong triển khai

- **Reverse proxy:** tiếp nhận yêu cầu, chuyển tiếp tới ứng dụng phía sau – ẩn máy chủ gốc, tăng bảo mật.
- **Load balancing:** phân phối lưu lượng đều cho nhiều máy chủ, tự động bỏ qua máy chủ gặp sự cố.
- **Caching:** lưu bản sao phản hồi, trả kết quả ngay mà không cần gọi lại ứng dụng.
- **SSL termination:** giải mã HTTPS ngay tại Nginx, giúp ứng dụng phía sau chỉ cần xử lý HTTP thường.

# Vì sao cần SSL/TLS cho máy chủ web?

- Không có SSL/TLS: dữ liệu truyền dạng văn bản thuần túy – dễ bị tấn công  
**Man-in-the-Middle.**
- TLS là phiên bản kế thừa, hiện đại hơn SSL; các bản SSL cũ đã bị khai tử.
- “Chứng chỉ SSL” ngày nay thực chất đang dùng giao thức TLS (1.2, 1.3).



# Ba giá trị cốt lõi của SSL/TLS

- **Mã hóa (Encryption):** biến dữ liệu thành chuỗi vô nghĩa với người không có khóa giải mã.
- **Xác thực danh tính (Authentication):** chứng chỉ do CA (Certificate Authority) cấp, xác nhận đúng chủ sở hữu tên miền.
- **Toàn vẹn dữ liệu (Integrity):** phát hiện và ngắt kết nối nếu dữ liệu bị chỉnh sửa trên đường truyền.

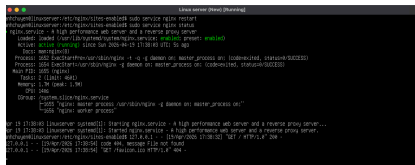
## Lợi ích khác

HTTPS còn cải thiện thứ hạng SEO và tránh cảnh báo “Không an toàn” trên trình duyệt.

# Cài đặt Nginx trên máy chủ Linux

```
$sudo apt-get update
$sudo apt-get install nginx
$sudo nginx -v
```

- Cấu hình lưu tại `/etc/nginx/`.
- Tập chính: `nginx.conf`, thư mục `sites-available`.



```
Linux server (New) [running]
shahyem@linuxserver:~$sudo service nginx restart
nginx.service - A high performance web server and a reverse proxy server
loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
Active: active (running) since Sun 2020-04-19 17:38:48 UTC; 5s ago
Docs: http://nginx.org/
Process: 2652 ExecStartPre=/usr/sbin/nginx -t -q & daemon on; master process on; (code=exited, status=SUCCESS)
Process: 2654 ExecStart=/usr/sbin/nginx -q daemon on; master process on; (code=exited, status=SUCCESS)
Main PID: 2655 (nginx)
Tasks: 2 (limit 4841)
Memory: 3.3M (max=6.5M)
CPU: 346s
CGroup: /system.slice/nginx.service
└─2655 "nginx: master process /usr/sbin/nginx -q daemon on; master process on;"
    └─2656 "nginx: worker process"

Apr 19 17:38:48 linuxserver systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Apr 19 17:38:48 linuxserver systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
shahyem@linuxserver:~$sudo service nginx status
nginx.service is active (running) since Sun 2020-04-19 17:38:48 UTC; 5s ago; 127.0.0.1 - [19/Apr/2020:17:38:52] "GET / HTTP/1.1" 200 -
127.0.0.1 - [19/Apr/2020:17:38:54] 200 404, response file not found
127.0.0.1 - [19/Apr/2020:17:38:54] "GET /favicon.ico HTTP/1.1" 404 -
```

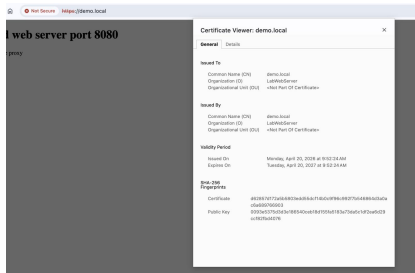
# Quy trình thực hành: Reverse Proxy + HTTPS tự ký

- 1 Cài Nginx và OpenSSL; khai báo domain thử nghiệm trong `/etc/hosts`.
- 2 Chạy một backend đơn giản (ví dụ Python HTTP server) trên cổng nội bộ.
- 3 Cấu hình Nginx làm reverse proxy chuyển tiếp về backend qua HTTP (cổng 80).
- 4 Tạo chứng chỉ tự ký bằng `openssl req -x509` (khóa RSA 2048-bit).
- 5 Bật HTTPS trong Nginx: khai báo `ssl_certificate`, chuyển hướng HTTP sang HTTPS.
- 6 Kiểm tra bằng `nginx -t`, `systemctl status`, `journalctl -xe`.

# Kết quả: website hoạt động với HTTPS tự ký



Website chạy qua HTTPS



Thông tin chứng chỉ SSL/TLS

# Tổng kết chương

## Những điểm cần ghi nhớ

- DevOps là văn hóa Dev + Ops theo vòng lặp 7 giai đoạn, không phải một công cụ.
- IaC chuyển cấu hình hạ tầng từ thủ công sang khai báo, đảm bảo nhất quán và có thể tái bản.
- Linux là nền tảng của Cloud/DevOps; cấu hình dịch vụ theo 4 bước: cài đặt – cấu hình – kiểm soát – kiểm tra.
- Nginx (event-driven) đảm nhiệm reverse proxy, cân bằng tải, cache và SSL termination.
- SSL/TLS bảo đảm mã hóa, xác thực và toàn vẹn dữ liệu; chứng chỉ tự ký phù hợp môi trường học tập.
- Đã triển khai thành công Nginx reverse proxy kèm HTTPS tự ký, biết dùng `systemctl/journalctl` để chẩn đoán lỗi.